omp-course-lab - Service Specification (v2)

This document describes the behaviour of the lab service as shipped on `main`.
`docs/spec.pdf` is generated from this file by `tools/make_pdf.py` and is the
fixture for the PDF-reading exercise; keep the two in sync by re-running the tool.

1. Overview

The lab is a deliberately small order-management backend:

- `api/` - a JSON HTTP service (stdlib `http.server`) on port 8080 that also
  serves the static signup page from `web/`.
- `cli/` - an argparse client that reads the same SQLite database directly.
- `data/lab.sqlite` - the seeded database (12 users, 72 orders, 8 months).

Everything is Python 3 standard library. There is no dependency to install.

2. Data model

users

column	type	notes
id	INTEGER	primary key
name	TEXT	display name
email	TEXT	unique, lower-case
created_at	TEXT	ISO date `YYYY-MM-DD`
company	TEXT	optional, nullable (v2)

orders

column	type	notes
id	INTEGER	primary key
user_id	INTEGER	references users.id
created_at	TEXT	ISO date `YYYY-MM-DD`
status	TEXT	one of `pending`, `shipped`, `completed`, `cancelled`
total_cents	INTEGER	order total in cents (never floating point)

schema_version

Single row, `version = 2`. Version 1 databases are upgraded in place by
`python3 tools/migrate.py [--db PATH]`, which adds `users.company` and bumps the
version; running it again prints `already at version 2` and changes nothing.
Fresh databases from `tools/seed_db.py` are created at version 2. The constants
derived from the schema live in `generated/schema_info.py`, which is written by
`tools/seed_db.py` and must never be edited by hand.

3. HTTP API

All responses are JSON. Errors have the shape `{"error": "<message>"}`.

GET /health

Returns `{"status": "ok", "service": "...", "version": "1.0"}`.

GET /users/<id>

Returns the user record plus `company` (`null` when unset) and `order_count`, the number of the user's orders whose status is not `cancelled`.

```
{"id": 3, "name": "Linus Torvalds", "email": "linus@example.com",
 "created_at": "2025-09-30", "company": null, "order_count": 5}
```

`404` when the id does not exist.

GET /users/<id>/orders?status=<status>

All of one user's orders, newest first, **without pagination**. Each order has the same shape as in `/orders`. `status` is optional; when present it must be one of the values in `generated.schema_info.STATUSES`, otherwise `400` with an error naming the valid values. `404` when the user does not exist.

```
{"user_id": 9, "count": 2,
 "orders": [{"id": 61, "user_id": 9, "created_at": "2026-04-05",
             "status": "cancelled", "total_cents": 20299}, ...]}
```

GET /orders?month=YYYY-MM&page=N

Lists orders newest first, ten per page (`page` defaults to 1). `month` is optional; when present it must be `YYYY-MM` (`400` otherwise). The response carries `count`, the total number of matching orders across all pages, so that `ceil(count / page_size)` is the last page.

```
{"month": "2026-03", "page": 1, "page_size": 10, "count": 12,
 "orders": [{"id": 56, "user_id": 9, "created_at": "2026-03-23",
             "status": "pending", "total_cents": 338990}, ...]}
```

POST /signup

Body: `{"name": "...", "email": "...", "company": "..."}` (`company` optional). Creates a user and returns `201` with `{"id", "name", "email", "company"}`; `company` is `null` when omitted or blank. Intended behaviour for bad input:

- missing or malformed field -> `400`
- duplicate email -> `409`
- anything else (database unavailable, bug) -> `500`

The static page at `/` posts to this endpoint (name, email and the optional company field) and shows the banner "Welcome aboard!" on success.

Static files

`GET /` serves `web/index.html`; other paths fall through to files under `web/`.
Paths outside `web/` are rejected with `404`.

4. Command-line client

```
python -m cli users <id>                                    # one user + order count
python -m cli orders [--month YYYY-MM] [--status STATUS]  # order table, newest first
python -m cli health                                        # database summary
```

`--month` and `--status` combine with AND. `--status` accepts only the values in `generated.schema_info.STATUSES`; anything else is an argparse usage error (exit `2`).

Money is always rendered from integer cents as `$12.34`. The CLI exits `0` on success, `1` for "not found", `2` for a `LabError` (configuration problems), and argparse's `2` for usage errors.

5. Configuration

variable	meaning	default
`DATABASE_URL`	path or `sqlite:///path` to the database	`data/lab.sqlite`
`LAB_TOKEN`	reserved for a future auth exercise	unset
`LAB_ISSUE`	enables the failing test for issue *N*	unset

`.env.example` shows the expected shapes. The code never reads `.env` itself.

6. Testing

`python -m unittest discover -s tests` runs the whole suite in well under ten seconds. Tests copy the seeded database to a temporary file and point `DATABASE_URL` at the copy, so they never modify `data/lab.sqlite`.

Each seeded issue in `docs/ISSUES.md` has a test that is skipped unless `LAB_ISSUE=<n>` is set; those tests fail on `main` by design and pass once the issue is fixed.

7. Non-goals

- No authentication, no TLS, no rate limiting.
- No ORM, no migrations framework: `tools/migrate.py` is the one hand-written migration.
- The service is single-node and keeps no state outside SQLite.